

Project 74: Melbourne Urban Forest Canopy Analysis (2008-2021)

Student: Bramuel Bukhuni

Unit: SIT764 Team Project (A) - Trimester 1 2026

Project: Analyzing tree canopy coverage evolution across Melbourne

Objective

Analyzing tree canopy datasets from 2008, 2011, 2016, and 2021 to:

- Quantify canopy coverage changes over 13 years
- Identify growth and loss hotspots
- Forecast future canopy trends (2026, 2030)
- Support Melbourne's climate adaptation goals

Step 1: Importing Libraries

```
In [4]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

print("✓ Libraries loaded")
```

```
✓ Libraries loaded
```

Step 2: Loading Datasets from Local Files

```
In [6]: # Loading all four datasets
dataset_2008 = pd.read_csv('../data/tree-canopies-2008-urban-forest.csv')
print(f"2008: {len(dataset_2008):,} rows × {len(dataset_2008.columns)} columns")

dataset_2011 = pd.read_csv('../data/tree-canopies-2011-urban-forest.csv')
print(f"2011: {len(dataset_2011):,} rows × {len(dataset_2011.columns)} columns")

dataset_2016 = pd.read_csv('../data/tree-canopies-2016-urban-forest.csv')
print(f"2016: {len(dataset_2016):,} rows × {len(dataset_2016.columns)} columns")

dataset_2021 = pd.read_csv('../data/tree-canopies-2021-urban-forest.csv')
print(f"2021: {len(dataset_2021):,} rows × {len(dataset_2021.columns)} columns")
```

```
2008: 58,790 rows × 7 columns
2011: 94,699 rows × 51 columns
2016: 59,885 rows × 3 columns
2021: 57,980 rows × 2 columns
```

```
✓ All datasets loaded
```

Step 3: Inspecting the Dataset Structure (2008)

```
In [7]: # Check structure of 2008 dataset
print("DATASET STRUCTURE - 2008")
print("="*70)
print(f"Rows: {len(dataset_2008):,}")
print(f"Columns: {len(dataset_2008.columns)}")
print("\nColumn names:")
for i, col in enumerate(dataset_2008.columns, 1):
    print(f"  {i}. {col}")

dataset_2008.head()
```

DATASET STRUCTURE - 2008

=====

Rows: 58,790

Columns: 7

Column names:

1. Geo Point
2. Geo Shape
3. updated_by
4. shape_area
5. data_linea
6. shape_leng
7. updated_da

Out[7]:

	Geo Point	Geo Shape	updated_by
0	-37.81976781729536, 144.94441801188927	{"coordinates": [[[[144.94441865724306, -37.81...	Grace Detailed- GIS Services
1	-37.806110716056814, 144.9503206954275	{"coordinates": [[[[144.95033282662382, -37.80...	Grace Detailed- GIS Services

	Geo Point	Geo Shape	updated_by
2	-37.8203253438025, 144.97620406411045	{"coordinates": [[[[144.97619825473942, -37.82...	Grace Detailed- GIS Services
3	-37.8203038644944, 144.97690235601573	{"coordinates": [[[[144.97691449320791, -37.82...	Grace Detailed- GIS Services
4	-37.819748746210614, 144.94763298258226	{"coordinates": [[[[144.94763309162377, -37.81...	Grace Detailed- GIS Services

In [8]: # 2008 Dataset – Complete Overview

```
# Shape
print("Shape:")
display(dataset_2008.shape)

# Columns
print("\nColumns:")
display(dataset_2008.columns)

# Data types
print("\nData Types:")
display(dataset_2008.dtypes)

# Missing values
print("\nMissing Values:")
display(dataset_2008.isnull().sum())

# First 5 rows
print("\nFirst 5 Rows:")
display(dataset_2008.head())

# Summary statistics
print("\nSummary Statistics:")
display(dataset_2008.describe())
```

Shape:

(58790, 7)

Columns:

```
Index(['Geo Point', 'Geo Shape', 'updated_by', 'shape_area',
      'data_linea',
      'shape_leng', 'updated_da'],
      dtype='object')
```

Data Types:

```
Geo Point      object
Geo Shape      object
updated_by     object
shape_area     float64
data_linea     object
shape_leng     float64
updated_da     object
dtype: object
```

Missing Values:

Geo Point 0
Geo Shape 0
updated_by 69
shape_area 0
data_linea 69
shape_leng 0
updated_da 69
dtype: int64

First 5 Rows:

	Geo Point	Geo Shape	updated_by
0	-37.81976781729536, 144.94441801188927	{"coordinates": [[[144.94441865724306, -37.81...	Grace Detailed- GIS Services
1	-37.806110716056814, 144.9503206954275	{"coordinates": [[[144.95033282662382, -37.80...	Grace Detailed- GIS Services
2	-37.8203253438025, 144.97620406411045	{"coordinates": [[[144.97619825473942, -37.82...	Grace Detailed- GIS Services
3	-37.8203038644944, 144.97690235601573	{"coordinates": [[[144.97691449320791, -37.82...	Grace Detailed- GIS Services
4	-37.819748746210614, 144.94763298258226	{"coordinates": [[[144.94763309162377, -37.81...	Grace Detailed- GIS Services

	Geo Point	Geo Shape	updated_by

Summary Statistics:

	shape_area	shape_leng
count	58790.000000	58790.000000
mean	73.115561	25.587987
std	1098.342572	149.536748
min	0.000005	0.009742
25%	3.819509	7.188098
50%	10.749105	12.409316
75%	38.144451	24.202594
max	200251.406934	27230.264900

Step 4: Dataset Exploration - 2011

```
In [9]: # 2011 Dataset – Complete Overview

# Shape
print("Shape:")
display(dataset_2011.shape)

# Columns
print("\nColumns:")
display(dataset_2011.columns)

# Data types
print("\nData Types:")
display(dataset_2011.dtypes)

# Missing values
print("\nMissing Values:")
display(dataset_2011.isnull().sum())

# First 5 rows
print("\nFirst 5 Rows:")
display(dataset_2011.head())

# Summary statistics
print("\nSummary Statistics:")
display(dataset_2011.describe())
```

Shape:

(94699, 51)

Columns:

```
Index(['Geo Point', 'Geo Shape', 'tree_area_', 'street_fro',
      'data_lin_1',
      'yearplant', 'street_nam', 'updated_by', 'shape_area',
      'park_stree',
      'location', 'scientific', 'height_112', 'admin_name',
      'data_linea',
      'ule', 'native', 'site', 'easting', 'ule_value',
      'sub_site', 'northing',
      'evergreen', 'gis_id', 'objectid', 'common_nam',
      'age', 'str_id',
      'shape_leng', 'condition_', 'updated_da',
      'height_111', 'precinct',
      'genus', 'assetid', 'street_id', 'mcc_id',
      'address_po', 'dbh',
      'admin_id', 'roadseg_id', 'street_to', 'family',
      'ggis_id',
      'overhead_c', 'date_plant', 't11', 'canopy_dia',
```

```
'height_11_',
      'descriptio', 'roadseg_de'],
      dtype='object')
```

Data Types:

Geo Point	object
Geo Shape	object
tree_area_	float64
street_fro	object
data_lin_1	object
yearplant	int64
street_nam	object
updated_by	object
shape_area	float64
park_stree	object
location	object
scientific	object
height_112	float64
admin_name	float64
data_linea	object
ule	object
native	object
site	object
easting	float64
ule_value	int64
sub_site	float64
northing	float64
evergreen	object
gis_id	int64
objectid	int64
common_nam	object
age	object
str_id	float64
shape_leng	float64
condition_	int64
updated_da	object
height_111	float64
precinct	object
genus	object
assetid	int64
street_id	object
mcc_id	float64
address_po	object
dbh	int64
admin_id	int64
roadseg_id	int64
street_to	object
family	object
ggis_id	int64
overhead_c	object
date_plant	object
t11	int64
canopy_dia	float64
height_11_	float64
descriptio	object

```
roadseg_de    object
dtype: object
```

Missing Values:

Geo Point	0
Geo Shape	0
tree_area_	0
street_fro	75679
data_lin_1	0
yearplant	0
street_nam	76014
updated_by	0
shape_area	0
park_stree	63035
location	61264
scientific	56606
height_112	0
admin_name	94699
data_linea	0
ule	68129
native	60970
site	69768
easting	0
ule_value	0
sub_site	94699
northing	0
evergreen	60970
gis_id	0
objectid	0
common_nam	60556
age	68176
str_id	60765
shape_leng	0
condition_	0
updated_da	0
height_111	0
precinct	56598
genus	56611
assetid	0
street_id	75397
mcc_id	0
address_po	60765
dbh	0
admin_id	0
roadseg_id	0
street_to	76099
family	56607
ggis_id	0
overhead_c	60767
date_plant	63663
t11	0
canopy_dia	0
height_11_	0
descriptio	56598

roadseg_de 75399
dtype: int64

First 5 Rows:

	Geo Point	Geo Shape	tree_area_	s
0	-37.7971025448064, 144.92805294520574	{"coordinates": [[[[144.92808097355524, -37.79...	85.552695	⤴
1	-37.79677607198214, 144.92647587608803	{"coordinates": [[[[144.92650356294064, -37.79...	30.053766	⤴
2	-37.79615547394656, 144.93082811899046	{"coordinates": [[[[144.93088560280367, -37.79...	63.268135	V F
3	-37.79686430258301, 144.92822217520853	{"coordinates": [[[[144.92829641122773, -37.79...	129.212831	E S
4	-37.79531120478856, 144.92467733916294	{"coordinates": [[[[144.92465442974685, -37.79...	25.217355	⤴

5 rows × 51 columns

Summary Statistics:

	tree_area_	yearplant	shape_area	height_
count	94699.000000	94699.000000	94699.000000	94699.000
mean	43.316229	660.443078	43.316229	5.610190
std	65.253746	940.969068	65.253746	4.958371
min	0.005476	0.000000	0.005476	0.000000
25%	4.902688	0.000000	4.902688	2.000000
50%	18.487083	0.000000	18.487083	4.000000
75%	53.964227	1997.000000	53.964227	8.000000
max	2429.758076	2012.000000	2429.758076	24.000000

8 rows × 24 columns

Step 5: Dataset Exploration - 2016

```
In [10]: # 2016 Dataset – Complete Overview

# Shape
print("Shape:")
display(dataset_2016.shape)

# Columns
print("\nColumns:")
display(dataset_2016.columns)

# Data types
print("\nData Types:")
display(dataset_2016.dtypes)

# Missing values
print("\nMissing Values:")
display(dataset_2016.isnull().sum())

# First 5 rows
print("\nFirst 5 Rows:")
display(dataset_2016.head())

# Summary statistics
print("\nSummary Statistics:")
display(dataset_2016.describe())
```

Shape:

(59885, 3)

Columns:

Index(['Geo Point', 'Geo Shape', 'area'], dtype='object')

Data Types:

Geo Point	object
Geo Shape	object
area	float64
dtype:	object

Missing Values:

Geo Point	0
Geo Shape	0

area0
dtype: int64

First 5 Rows:

	Geo Point	Geo Shape	area
0	-37.8258826364104, 144.90891093468164	{"coordinates": [[[144.9089448552414, -37.8258...	26.980360
1	-37.82694480262385, 144.96757472864104	{"coordinates": [[[144.96760279483098, -37.826...	46.024143
2	-37.826707481062904, 144.95559181283596	{"coordinates": [[[144.9556154092041, -37.8266...	117.141605
3	-37.826676071951375, 144.95496514813638	{"coordinates": [[[144.95499949034675, -37.826...	51.855034
4	-37.82663523205097, 144.95466868431407	{"coordinates": [[[144.9546692404478, -37.8266...	11.139799

Summary Statistics:

	area
count	59885.000000
mean	72.346082
std	628.897991
min	0.001230
25%	3.217627
50%	12.309994
75%	48.413594
max	79768.567688

Step 6: Dataset Exploration - 2021

```
In [11]: # 2021 Dataset – Complete Overview

# Shape
print("Shape:")
display(dataset_2021.shape)

# Columns
print("\nColumns:")
display(dataset_2021.columns)

# Data types
print("\nData Types:")
display(dataset_2021.dtypes)

# Missing values
print("\nMissing Values:")
display(dataset_2021.isnull().sum())

# First 5 rows
print("\nFirst 5 Rows:")
display(dataset_2021.head())

# Summary statistics
print("\nSummary Statistics:")
display(dataset_2021.describe())
```

Shape:

(57980, 2)

Columns:

Index(['Geo Point', 'Geo Shape'], dtype='object')

Data Types:

Geo Point object
Geo Shape object
dtype: object

Missing Values:

Geo Point 0
Geo Shape 0
dtype: int64

First 5 Rows:

	Geo Point	Geo Shape
0	-37.8298681237421, 144.98303001088595	{"coordinates": [[[[144.9832974445821, -37.829...
1	-37.829874533279096, 144.97144661356745	{"coordinates": [[[[144.9714529379414, -37.829...
2	-37.83021396760069, 144.98646678135142	{"coordinates": [[[[144.98647926050035, -37.83...
3	-37.828742240015515, 144.9011718210025	{"coordinates": [[[[144.90116683929529, -37.82...
4	-37.829920930428415, 144.96518349051888	{"coordinates": [[[[144.96517363556384, -37.82...

Summary Statistics:

	Geo Point	Geo Shape
count	57980	57980
unique	57980	57980
top	-37.8298681237421, 144.98303001088595	{"coordinates": [[[[144.9832974445821, -37.829...
freq	1	1

Step 7: Column Consistency Check Across All Years

Verifying that all datasets have identical column structure for compatibility

In
[12]:

```
# Check column consistency

cols_2008 = set(dataset_2008.columns)
cols_2011 = set(dataset_2011.columns)
cols_2016 = set(dataset_2016.columns)
cols_2021 = set(dataset_2021.columns)

print("Column Consistency Check:")
print("="*70)

if cols_2008 == cols_2011 == cols_2016 == cols_2021:
    print("✓ All datasets have identical columns")
    print(f"\nTotal columns: {len(dataset_2008.columns)}")
else:
    print("△ Column differences detected\n")

    if cols_2008 != cols_2011:
        print("2008 vs 2011:")
        print(f"  Only in 2008: {cols_2008 - cols_2011}")
        print(f"  Only in 2011: {cols_2011 - cols_2008}\n")

    if cols_2008 != cols_2016:
        print("2008 vs 2016:")
        print(f"  Only in 2008: {cols_2008 - cols_2016}")
        print(f"  Only in 2016: {cols_2016 - cols_2008}\n")

    if cols_2008 != cols_2021:
        print("2008 vs 2021:")
        print(f"  Only in 2008: {cols_2008 - cols_2021}")
        print(f"  Only in 2021: {cols_2021 - cols_2008}")
```

Column Consistency Check:

=====

△ Column differences detected

2008 vs 2011:

Only in 2008: set()

Only in 2011: {'ule', 'street_nam', 'native', 'descriptio',
'site', 'family', 'tree_area_', 'assetid', 'admin_id',
'street_to', 'scientific', 'height_112', 'common_nam',
'date_plant', 'gis_id', 'ggis_id', 'age', 'condition_',
'easting', 'data_lin_1', 't11', 'height_111', 'roadseg_id',
'mcc_id', 'admin_name', 'roadseg_de', 'dbh', 'street_fro',
'precinct', 'park_stree', 'canopy_dia', 'northing',
'ule_value', 'evergreen', 'objectid', 'sub_site', 'str_id',
'genus', 'yearplant', 'address_po', 'overhead_c', 'location',
'street_id', 'height_11_'}

2008 vs 2016:

Only in 2008: {'shape_leng', 'updated_by', 'updated_da',
'shape_area', 'data_linea'}

Only in 2016: {'area'}

2008 vs 2021:

Only in 2008: {'shape_leng', 'updated_by', 'updated_da',

```
'shape_area', 'data_linea'}
Only in 2021: set()
```

Step 8: Identify Area Column in Each Dataset

Find which column contains canopy area information

```
In [13]: # Check for area-related columns in each dataset

print("Area-related columns in each dataset:")
print("="*70)

# 2008
area_cols_2008 = [col for col in dataset_2008.columns
                  if 'area' in col.lower() or 'shape' in
col.lower()]
print(f"\n2008: {area_cols_2008}")

# 2011
area_cols_2011 = [col for col in dataset_2011.columns
                  if 'area' in col.lower() or 'shape' in
col.lower()]
print(f"2011: {area_cols_2011}")

# 2016
area_cols_2016 = [col for col in dataset_2016.columns
                  if 'area' in col.lower() or 'shape' in
col.lower()]
print(f"2016: {area_cols_2016}")

# 2021
area_cols_2021 = [col for col in dataset_2021.columns
                  if 'area' in col.lower() or 'shape' in
col.lower()]
print(f"2021: {area_cols_2021}")
```

Area-related columns in each dataset:

```
=====
=====
```

```
2008: ['Geo Shape', 'shape_area', 'shape_leng']
2011: ['Geo Shape', 'tree_area_', 'shape_area', 'shape_leng']
2016: ['Geo Shape', 'area']
2021: ['Geo Shape']
```

Step 9: Examine Area Columns

Check sample values to understand the data structure

In
[14]:

```
# Check area column values for each year

print("Sample area values:")
print("="*70)

# 2008
print("\n2008 - shape_area column:")
display(dataset_2008['shape_area'].head())
display(dataset_2008['shape_area'].describe())

# 2011
print("\n2011 - tree_area_ column:")
display(dataset_2011['tree_area_'].head())
display(dataset_2011['tree_area_'].describe())

# 2016
print("\n2016 - area column:")
display(dataset_2016['area'].head())
display(dataset_2016['area'].describe())

# 2021 - check if there's a geometry column we can calculate area
from
print("\n2021 - Available columns:")
display(dataset_2021.columns.tolist())
```

Sample area values:

=====

2008 - shape_area column:

```
0      2.735103
1     21.070870
2     21.010537
3      6.751041
4      3.098483
Name: shape_area, dtype: float64
```

```
count      58790.000000
mean         73.115561
std       1098.342572
min           0.000005
25%          3.819509
50%         10.749105
75%         38.144451
max       200251.406934
Name: shape_area, dtype: float64
```

2011 - tree_area_ column:

```
0      85.552695
1     30.053766
2     63.268135
3    129.212831
```

```
4      25.217355
```

```
Name: tree_area_, dtype: float64
```

```
count      94699.000000
```

```
mean        43.316229
```

```
std         65.253746
```

```
min         0.005476
```

```
25%         4.902688
```

```
50%        18.487083
```

```
75%        53.964227
```

```
max        2429.758076
```

```
Name: tree_area_, dtype: float64
```

```
2016 – area column:
```

```
0      26.980360
```

```
1      46.024143
```

```
2     117.141605
```

```
3      51.855034
```

```
4      11.139799
```

```
Name: area, dtype: float64
```

```
count      59885.000000
```

```
mean        72.346082
```

```
std         628.897991
```

```
min         0.001230
```

```
25%         3.217627
```

```
50%        12.309994
```

```
75%        48.413594
```

```
max       79768.567688
```

```
Name: area, dtype: float64
```

```
2021 – Available columns:
```

```
['Geo Point', 'Geo Shape']
```

Key Findings from Exploration

Dataset Structures:

- **2008:** 58,790 canopy polygons, area in `shape_area` (square meters)
- **2011:** 94,699 tree records, area in `tree_area_` (square meters)
- **2016:** 59,885 canopy polygons, area in `area` (square meters)
- **2021:** Unknown count, only has `Geo Shape geometry` - **area needs to be calculated**

Next Steps:

1. Calculate area from geometry for 2021 dataset
2. Standardize area column name across all years
3. Calculate total canopy coverage per year
4. Analyze trends

Note: 2011 appears to be individual tree data (different from canopy polygon data in other years)

Week 1-2 Deliverable Complete

Completed Tasks:

- All datasets loaded successfully
- Data structure explored for each year
- Column differences identified
- Area columns located (2008, 2011, 2016)
- Issue identified: 2021 requires geometry-based area calculation

In []: